

**PROJET
ELECTRONIQUE**



MACHINE DE TURING

12/06/2026



**SAMBA NIANG
DYLAN DESHAUTEURS**

Sommaire

I.	Introduction.....	2
	Fondements Historiques	2
	La Machine de Turing Universelle (MTU)	2
II.	Contexte Académique & états des lieux	3
	Continuité pédagogique	3
	Synthèse des travaux antérieurs	3
III.	Objectifs du Projet (Cahier des charges)	4
	Phase 1 : Étude Théorique et Formalisation	4
	Phase 2 : Audit et Reprise de l'Existant	4
	Phase 3 : Développement et Intégration Matérielle	5
	Phase bonus : Packaging et Conception CAO	5
IV.	Analyse fonctionnelle	5
	Définition des besoins	5
	Bête à corne.....	5
	Matrice MoSCoW	6
	1. SFN1.....	6
	2. SF1D.....	7
	3. SF2D – FP2 (IHM_Entrée)	8
	4. SF2D – FP3 (IHM_Sortie)	8
	Spécifications des fonctions et de leurs signaux de communication	9
V.	Conception matérielle (Hardware).....	17
	Choix des composants.....	17
	Schéma structurel.....	19
	Conception du circuit imprimé (PCB)	21
VI.	Conception logicielle et algorithmique (Software)	23
	Logique de traitement.....	23
	Modélisation algorithmique.....	25
VII.	Intégration et Validation Expérimentale	27
	Protocoles de Test	27

Elaboration des codes de test unitaire & firmware final.....	28
Conclusion g�n�rale et perspectives.....	30
Remerciement.....	31
Annexe & bibliographie	32
Feuille de route du projet.....	32
Algorithme d'addition de deux nombres binaires.....	33
Fiche de test	34
Firmware de la machine de Turing	35

I. Introduction

Fondements Historiques

En 1936, le mathématicien britannique Alan Turing pose la pierre angulaire de l'informatique moderne avec son article « *On Computable Numbers* ». Loin des préoccupations matérielles de l'époque, Turing mène une réflexion de logique pure pour définir l'essence même du calcul.

Il conçoit alors une machine universelle théorique : un concept révolutionnaire capable d'exécuter n'importe quel processus logique, pourvu qu'il soit programmable.

Cette intuition théorique trouve son application concrète dès le déclenchement de la Seconde Guerre mondiale. Turing met ses concepts au service du décryptage pour briser les codes de la machine allemande Enigma.

La Machine de Turing Universelle (MTU)

Le concept de Turing repose se compose de quatre éléments :

- Le Ruban : Une mémoire potentiellement infinie divisée en cases, chacune contenant un symbole.
- La Tête de Lecture/Écriture : Un dispositif capable de lire le symbole d'une case, d'en écrire un nouveau et de se déplacer vers la gauche ou la droite.
- Le Registre d'État : Il stocke l'état interne de la machine à un instant T.
- La Table de Transition : Il constitue le programme de la machine. Elle définit les règles d'interaction entre les trois composants précédents.

Le fonctionnement repose sur une table de transition. Pour chaque configuration (État actuel, Symbole lu), la machine exécute une action selon une règle stricte :

1. Écrire un nouveau symbole dans la case.
2. Changer l'état interne de la machine.
3. Déplacer la tête de lecture (Gauche/Droite/Immobile).

Exemple de transition : Si la machine est à l'état "E1" et lit "0", elle écrit "1", passe à l'état "E2" et se déplace vers la droite.

II. Contexte Académique & états des lieux

Dans le cadre du cursus Informatique et Systèmes Intelligents Embarqués (ISIE), ce projet vise à transcrire les concepts abstraits de Turing en une réalisation concrète, aboutissant à la création d'une Machine de Turing Électronique.

Continuité pédagogique

Cinquième équipe à relever ce défi, notre mission s'inscrit dans une véritable continuité pédagogique. L'enjeu est de consolider les acquis des groupes précédents pour aboutir à une version finale aboutie et stabilisée. Pour ce faire, notre développement s'appuie principalement sur l'architecture de ALLEGRE-COMMINGES Clément et BROUARD Romain, tout en intégrant les optimisations apportées par l'équipe de BOLARD Yassine et MANSO Alexis.

Synthèse des travaux antérieurs

Le développement du projet a suivi une courbe d'apprentissage rigoureuse, partant de la conception théorique vers la mise en œuvre technique. Les premières recherches des groupes précédents ont permis de poser un socle fonctionnel solide.

L'effort s'est porté sur la production du firmware et l'assemblage des premiers composants. Toutefois, la réalisation finale a été freinée par des aléas matériels notamment des défaillances sur le ruban LED et par la complexité de l'intégration globale.

Ce passif technique nous permet aujourd'hui d'identifier pr cis ment les points critiques, tant sur la fiabilit  des composants que sur la gestion du temps, pour mener   bien l'ach vement de la maquette physique.

  notre arriv e, nous h ritons d'une analyse th orique solide. Le projet se trouvait toutefois inachev . Notre mission est d'agir comme int grateur final : diagnostiquer les d faillances mat rielles, corriger les erreurs de conception  lectronique et finaliser l'assemblage pour livrer une maquette 100% op rationnelle.

III. Objectifs du Projet (Cahier des charges)

L'objectif principal est la conception et la r alisation d'un prototype physique fonctionnel d'une Machine de Turing  lectronique, capable d'ex cuter des processus logiques programmables.

Phase 1 :  tude Th orique et Formalisation

L'enjeu est de ma triser les fondements logiques avant toute impl mentation technique :

1. D finir les principes de la machine de Turing et son architecture logique (Ruban, T te, Table de transition).
2.  tudier et formaliser les algorithmes de calcul associ s.
3. Pr senter et expliciter un cas d'usage simple (exemple : incr mentation binaire) pour valider la compr hension du mod le.

Phase 2 : Audit et Reprise de l'Existant

Afin de capitaliser sur les travaux des ann es pr c dentes, cette phase de "r tro-ing nierie" est cruciale :

1.  tudier les rapports existants pour en extraire les solutions les plus robustes.
2. Fixer l'architecture cible
3.  tablir la nomenclature des composants et lancer les commandes.

Phase 3 : Développement et Intégration Matérielle

Cette étape constitue le cœur technique de la réalisation :

1. Valider les schémas électroniques via des tests sur platine d'essai (Breadboard).
2. Adapter et implémenter le firmware en veillant à la stricte correspondance entre l'algorithme théorique et le déroulement du code.
3. Vérifier la fiabilité des cycles de lecture/écriture et des déplacements d'états.

Phase bonus : Packaging et Conception CAO

Afin d'aboutir à un produit fini et professionnel :

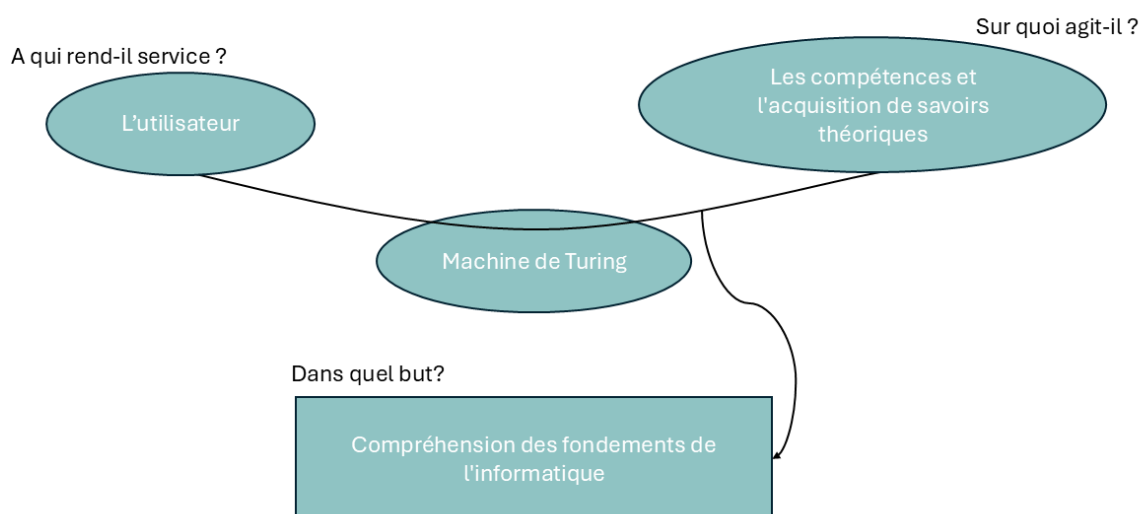
1. Réaliser un circuit imprimé (PCB).
2. Concevoir et fabriquer un boîtier pour protéger l'électronique.

La planification complète de ces différentes étapes est synthétisée au sein d'une feuille de route globale, disponible en Annexe.

IV. Analyse fonctionnelle

Définition des besoins

Bête à corne

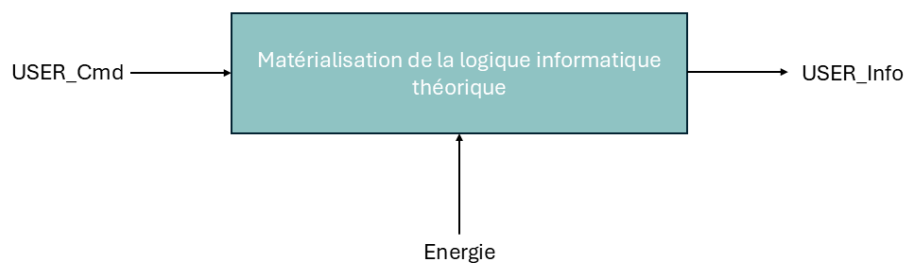


Matrice MoSCoW

Must Have	Should Have
- Navigation par boutons poussoir (Up/Down) - Représentation physique du ruban (Matrice LED RGB) - Signalisation des erreurs (LED Rouge) - Exécution d'une addition binaire	- Possibilité de faire d'autres opérations (soustraction, etc.) - Contrôle d'exécution (Boutons Start/Select) - Sélection des modes via un commutateur (Step_by_step ou Mode continue) - Affichage textuel de l'état sur un écran LCD
Could Have	Won't Have
	- Programmation sur mémoire externe (carte SD) - Saisie d'un programme directement sur la machine - Autonomie énergétique

1. SFN1

Afin de mieux appréhender l'interaction du système avec son environnement, nous avons établi un schéma fonctionnel de niveau 1.



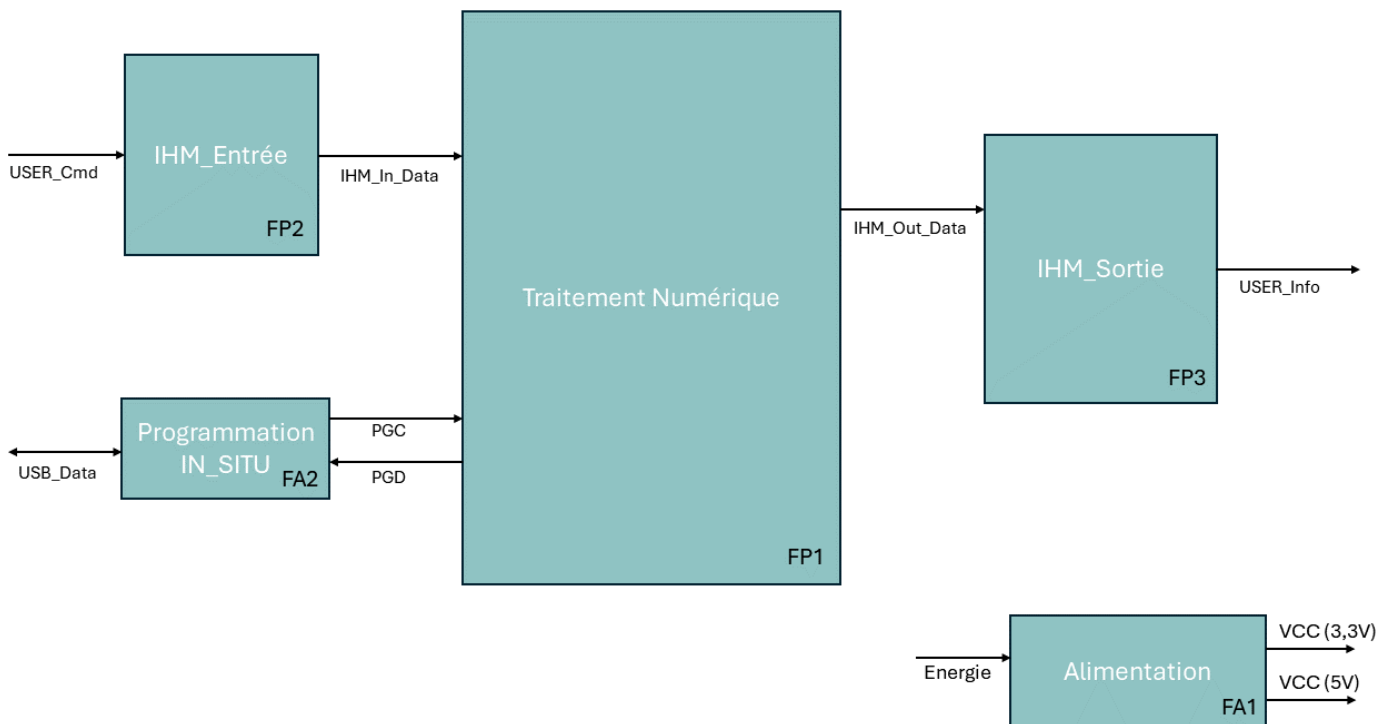
Les Commandes Utilisateur (boutons, commutateurs) pilotent la logique de la machine.

L'Énergie est fournie par le module d'alimentation stabilisé (3,3V/5V) pour permettre le fonctionnement des composants actifs.

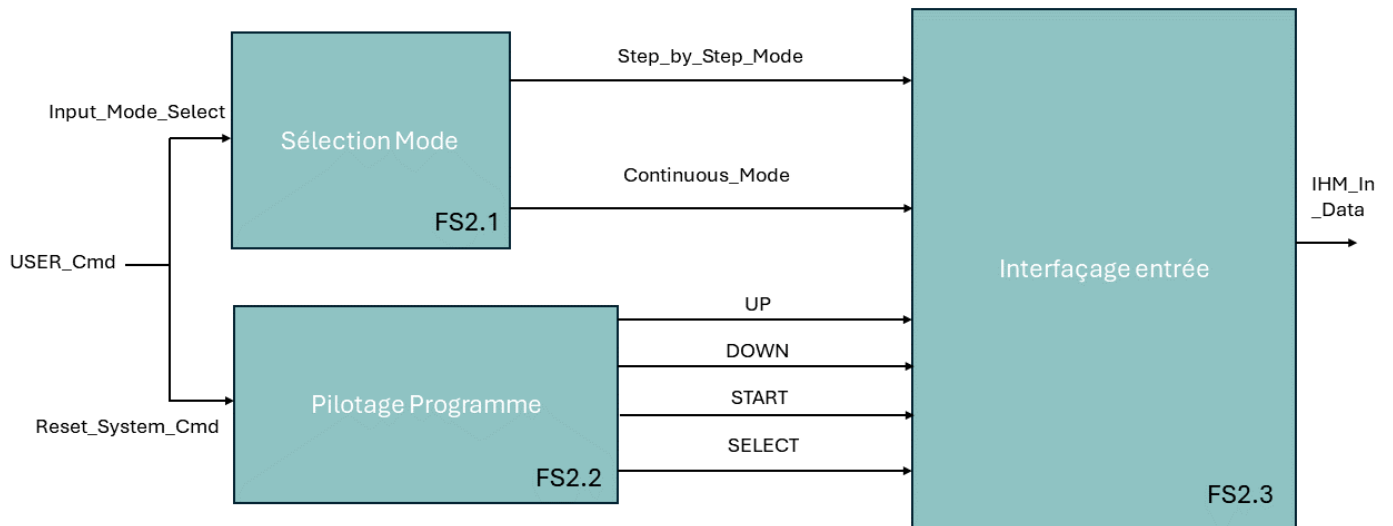
Le bloc Traitement de la donnée est assuré par le microcontrôleur PIC24, qui exécute les règles de transition.

Le Résultat se traduit par la mise à jour visuelle du ruban LED et de l'écran LCD.

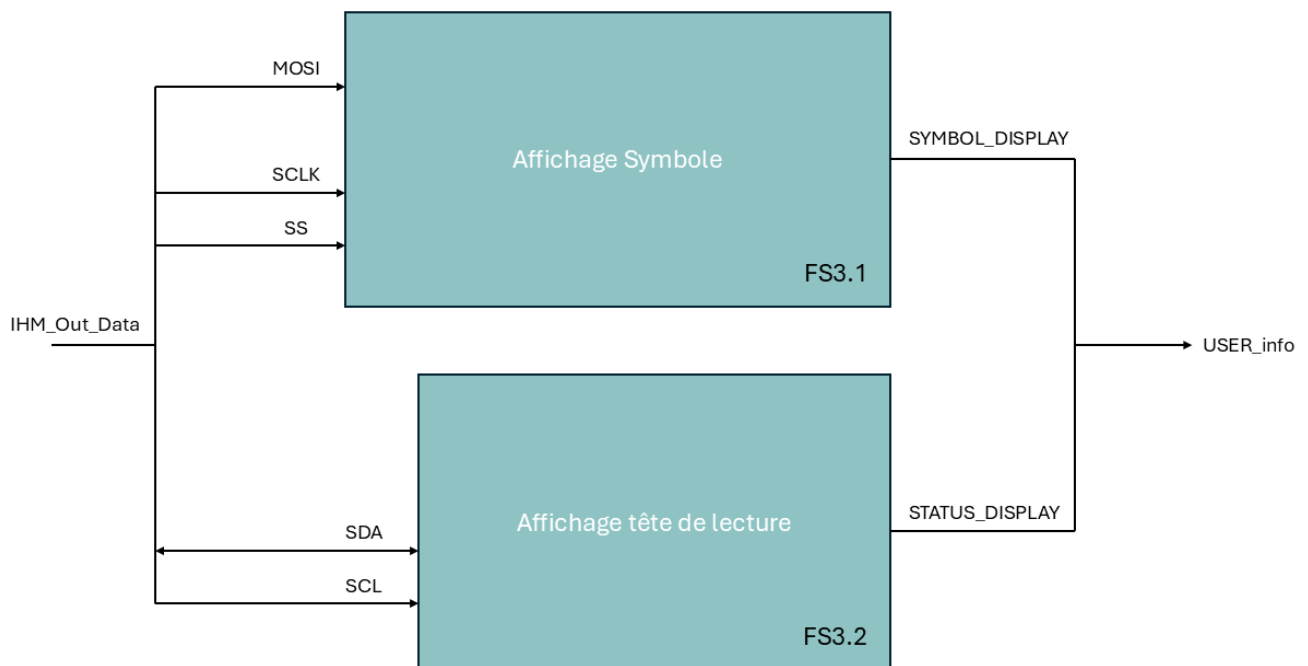
2. SF1D



3. SF2D – FP2 (IHM_Entrée)



4. SF2D – FP3 (IHM_Sortie)



Sp cifications des fonctions et de leurs signaux de communication

Fonction	FP1	Traitements Num�riques
Description / r�le	R�alise tous les traitements num�riques n�cessaires au bon fonctionnement du syst�me	
Signaux d'entr�e	IHM_In_Data, IHM_Out_Data	
Signaux de sortie	IHM_In_Ack, IHM_Out_St	
Signaux E/S	Prog_Data	

Fonction	FS 2.1	S�lection mode
Description / r�le	Fonction qui sert � s�lectionner le mode d'ex�cution pas � pas ou continu	
Signaux d'entr�e	Input_Mode_Select	
Signaux de sortie	Step_by_Step_Mode, Continuous_Mode	
Signaux E/S	NA	

Fonction	FS 2.2	Initialisation du programme
Description / r�le	Fonction qui permet d'initialiser les symboles et la t�te de lecture	
Signaux d'entr�e	Reset_System_Cmd	
Signaux de sortie	Start, Up, Down, Select	
Signaux E/S	NA	

Fonction	FS 2.3	Interfaçage d'entrée
Description / rôle	Fonction qui sert de réceptacle et de commande du MCU	
Signaux d'entrée	Start, Up, Down, Select, Step_by_Step_Mode, Continuous_Mode	
Signaux de sortie	SCL	
Signaux E/S	SDA	

Fonction	FS 3.1	Affichage Symbole
Description / rôle	Fonction réalisant l'affichage des symboles	
Signaux d'entrée	MOSI, SCLK, SS	
Signaux de sortie	MISO, Symbol_Display	
Signaux E/S	NA	

Fonction	FS 3.2	Affichage de tête de lecture
Description / rôle	Fonction réalisant l'affichage de la tête de lecture	
Signaux d'entrée	MOSI, SCLK, SS	
Signaux de sortie	MISO, Status_Display	
Signaux E/S	NA	

Fonction	FA 1	Alimentation
Description / rôle	Fonction qui alimente le système en électricité	
Signaux d'entrée	Energie	
Signaux de sortie	VCC	
Signaux E/S	NA	

Fonction	FA 2	Programmation In Situ
Description / rôle	Fonction permettant de charger le programme dans le MCU	
Signaux d'entrée	NA	
Signaux de sortie	NA	
Signaux E/S	USB_Data, Prog_Data	

Ce tableau détaille l'ensemble des caractéristiques physiques, électriques et logiques de tous les signaux du système.

Signal	Fonctions concernées	Nature (A/N/GP)	Taille entité	Grandeur & Unité	Plage de variation	Excursion fréq.	Valeur repos	Contraintes temp.	Norme
MISO	FP1, FS3.1, FS3.2	N	1	U (V)	Niveau Haut/Bas	NA	NA	NA	SPI
Description : Signal généré par l'esclave pour communiquer avec le maître.									

Signal	Fonctions concern�es	Nature (A/N/GP)	Taille entit�	Grandeur & Unit�	Plage de variation	Excursion fr�q.	Valeur repos	Contraintes temp.	Norme
MOSI	FP1, FS3.1, FS3.2	N	1	U (V)	Niveau Haut/Bas	NA	NA	NA	SPI
<i>Description : Signal g�n�r� par le ma�tre pour communiquer avec l'esclave.</i>									
SCLK	FP1, FS3.1, FS3.2	N	1	U (V)	Niveau Haut/Bas	NA	NA	NA	SPI
<i>Description : Signal de transmission de l'horloge par le ma�tre.</i>									
SS	FP1, FS3.1, FS3.2	N	1	U (V)	Niveau Haut/Bas	NA	NA	NA	SPI
<i>Description : Signal de s�lection de l'esclave par le ma�tre.</i>									
SDA	FP1, FS2.3	N	1	U (V)	Niveau Haut/Bas	NA	1	NA	I2C
<i>Description : Signal de transmission de donn�es.</i>									
SCL	FP1, FS2.3	N	1	U (V)	Niveau Haut/Bas	NA	1	NA	I2C
<i>Description : Signal de transmission de l'horloge.</i>									
User_Cmd	FS2.1, FS2.2	GP	NA	U (V)	NA	NA	NA	NA	NA
<i>Description : Action d'appui des boutons par l'utilisateur.</i>									

Signal	Fonctions concernées	Nature (A/N/GP)	Taille entité	Grandeur & Unité	Plage de variation	Excursion fréq.	Valeur repos	Contraintes temp.	Norme
Input_Mode_Select	FS2.1	N	1	U (V)	Niveau Haut/Bas	NA	NA	NA	NA
<i>Description : Image du mode : niveau bas = pas à pas, niveau haut = continu.</i>									
Reset_System_Cmd	FS2.2	N	1	U (V)	Niveau Haut/Bas	NA	NA	NA	NA
<i>Description : Image de l'appui sur le bouton d'initialisation.</i>									
Start	FS2.2	N	1	U (V)	Niveau Haut/Bas	NA	NA	NA	NA
<i>Description : Image de l'appui sur le bouton de lancement.</i>									
Energie	FA1	A	1	U (V)	230 V CA	50 Hz	NA	NA	NA
<i>Description : Courant alternatif monophasé (Réseau électrique).</i>									
VCC	FA1	A	1	U (V)	5 V ou 3.3 V CC	NA	NA	NA	NA
<i>Description : Énergie continue dont le MCU a besoin pour fonctionner.</i>									
ISCP	FP1, FA2	N	1	NA	Niveau Haut/Bas	NA	0	NA	NA
<i>Description : Signal de transmission des données pour la programmation in situ.</i>									

Signal	Fonctions concernées	Nature (A/N/GP)	Taille entité	Grandeur & Unité	Plage de variation	Excursion fréq.	Valeur repos	Contraintes temp.	Norme
Infos User	FS3.1, FS3.2	GP	NA	U (V)	NA	NA	NA	NA	NA
<i>Description : Affichage général combinant la tête de lecture et les symboles.</i>									
Display Sym	FS3.1	GP	NA	U (V)	NA	NA	NA	NA	NA
<i>Description : Affichage des symboles.</i>									
Display Tete	FS3.2	GP	NA	U (V)	NA	NA	NA	NA	NA
<i>Description : Affichage de la tête de lecture.</i>									
IHM_In_Data	FP1, FP2	N	À définir	U (V)	Niveau Haut/Bas	NA	NA	NA	Spécifique
<i>Description : Signal de données envoyé par l'interface d'entrée (FP2) vers l'unité de traitement numérique (FP1).</i>									
IHM_In_Ack	FP1, FP2	N	1	U (V)	Niveau Haut/Bas	NA	NA	NA	Spécifique
<i>Description : Signal d'acquiescement renvoyé par le traitement numérique (FP1) à l'interface d'entrée (FP2).</i>									
IHM_Out_Data	FP1, FP3	N	À définir	U (V)	Niveau Haut/Bas	NA	NA	NA	Spécifique
<i>Description : Signal contenant les données d'affichage envoyé par le traitement numérique (FP1) vers l'interface de sortie (FP3).</i>									

Signal	Fonctions concernées	Nature (A/N/GP)	Taille entité	Grandeur & Unité	Plage de variation	Excursion fréq.	Valeur repos	Contraintes temp.	Norme
IHM_Out_St	FP1, FP3	N	À définir	U (V)	Niveau Haut/Bas	NA	NA	NA	Spécifique
<i>Description : Signal d'état ou de contrôle renvoyé par l'interface d'affichage (FP3) vers le traitement numérique (FP1).</i>									
Prog_Data	FP1, FA2	N	À définir	U (V)	Niveau Haut/Bas	NA	NA	NA	Spécifique
<i>Description : Signal bidirectionnel d'échange de données de programmation entre le module In Situ (FA2) et le MCU (FP1).</i>									
USB_Data	FA2	N	À définir	U (V)	Spécifique USB	Horloge USB	NA	NA	USB
<i>Description : Liaison de données bidirectionnelle externe permettant de charger le programme via l'interface USB.</i>									
Step_by_Step_Mode	FS2.1, FS2.3	N	1	U (V)	Niveau Haut/Bas	NA	NA	NA	NA
<i>Description : Signal activant le mode d'exécution pas à pas envoyé à l'interfaçage d'entrée.</i>									
Continuous_Mode	FS2.1, FS2.3	N	1	U (V)	Niveau Haut/Bas	NA	NA	NA	NA
<i>Description : Signal activant le mode d'exécution continu envoyé à l'interfaçage d'entrée.</i>									
Up	FS2.2, FS2.3	N	1	U (V)	Niveau Haut/Bas	NA	NA	NA	NA

Signal	Fonctions concern�es	Nature (A/N/GP)	Taille entit�	Grandeur & Unit�	Plage de variation	Excursion fr�q.	Valeur repos	Contraintes temp.	Norme
<i>Description : Signal d'incr�mentation ou de d�placement vers le haut pour la configuration du syst�me.</i>									
Down	FS2.2, FS2.3	N	1	U (V)	Niveau Haut/Bas	NA	NA	NA	NA
<i>Description : Signal de d�cr�mentation ou de d�placement vers le bas pour la configuration du syst�me.</i>									
Select	FS2.2, FS2.3	N	1	U (V)	Niveau Haut/Bas	NA	NA	NA	NA
<i>Description : Signal de s�lection ou de validation d'une option envoy� � l'interfa�age d'entr�e.</i>									
SYMBOL_DISPLAY	FS3.1	GP	� d�finir	U (V)	Sp�cifique Matrice	NA	NA	NA	Matrice RGB
<i>Description : Signal physique de commande envoy� � la matrice de LED (RGB_MATRIX) pour afficher les symboles.</i>									
STATUS_DISPLAY	FS3.2	GP	� d�finir	U (V)	Sp�cifique �cran	NA	NA	NA	�cran LCD
<i>Description : Signal physique de commande envoy� � l'�cran LCD (LCD_SCREEN) pour afficher l'�tat de la t�te de lecture.</i>									

V. Conception mat rielle (Hardware)

Choix des composants

Par rapport aux objectifs initiaux et aux travaux des promotions pr c dentes, notre  quipe a pris la d cision de retirer plusieurs  l ments p riph riques, notamment le module de stockage externe par carte SD ainsi que les fonctions de programmation directe des tables de transition sur la machine.

Ce choix de simplification est dict  par une volont  de fiabiliser le c ur de calcul et de stabiliser les communications sur les bus I2C et SPI. En nous concentrant sur les fonctionnalit s essentielles, nous avons pu nous assurer que l'unit  centrale dispose de toutes les ressources n cessaires en m moire Flash interne pour ex cuter les algorithmes de Turing de mani re robuste. Ce recentrage technique nous permet ainsi de garantir une maquette fonctionnelle privil giant la qualit  de l'ex cution arithm tique   la multiplication d'options secondaires.

Pour transformer le mod le th orique en une maquette physique, nous avons s lectionn  des composants r pondant aux contraintes de robustesse et de simplicit  d'interfa age

Microcontr leur PIC24FJ64GA002

Le choix s'est port  sur ce microcontr leur 16 bits pour plusieurs raisons strat giques :

- Capacit  de calcul suffisante pour g rer les tables de transition et les op rations arithm tiques (addition, soustraction, ect).
- La m moire Flash interne est en accord avec notre choix de supprimer la carte SD, sa m moire interne permet de stocker directement les algorithmes.
- Il g re nativement les protocoles I2C pour le LCD et SPI pour la matrice LED

Interface Homme-Machine (IHM)

L'interface est compos e de :

- 4 Boutons Poussoirs qui assurent la navigation (Up/Down) et le contr le (Start/Select).
- 1 Commutateur   levier d di    la s lection mat rielle du mode (Continu ou Pas- -pas). Ce choix permet un changement d' tat franc.
- Une LED rouge pour signaler les erreurs syst me.

Dispositifs d'Affichage

- ** cran LCD (NHD-C0220BIZ) :**

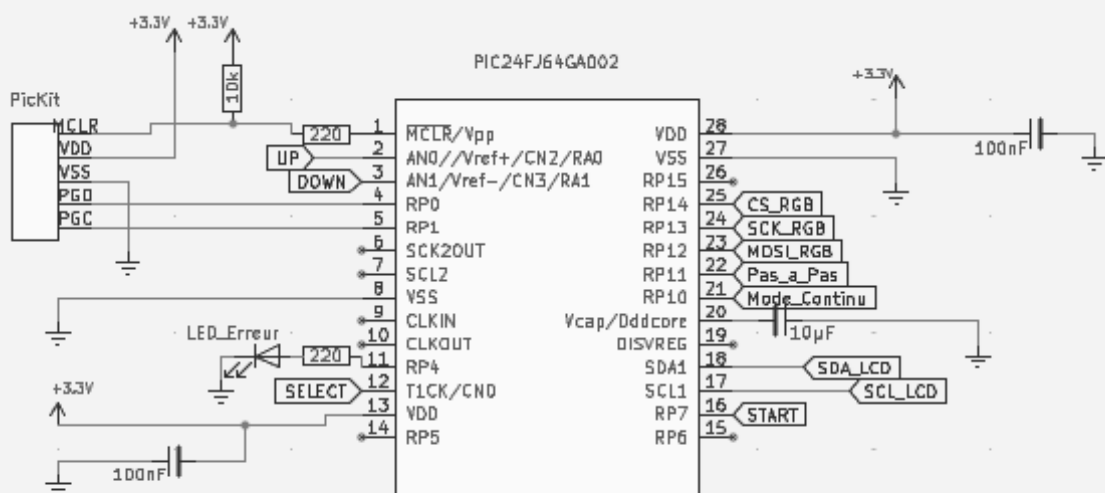
Sa fonction principale est l'affichage du menu, de l' tat interne de la machine et de la table de transition. Il utilise une interface par bus I2C afin de minimiser le nombre de fils n cessaires sur le PIC24.

- **Matrice de LEDs RGB :**

La matrice de LEDs RGB assure la repr sentation physique du Ruban de Turing. Dans cette configuration, chaque LED simule une case du ruban et change de couleur selon le symbole stock . L'interface utilise le bus SPI pour garantir une vitesse de rafra chissement  lev e, ce qui est particuli rement crucial lors des calculs effectu s en mode continu.

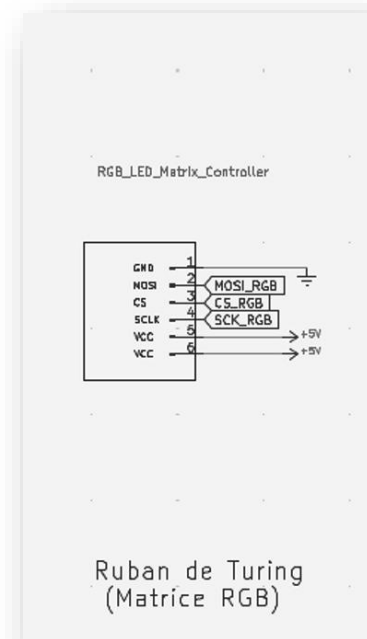
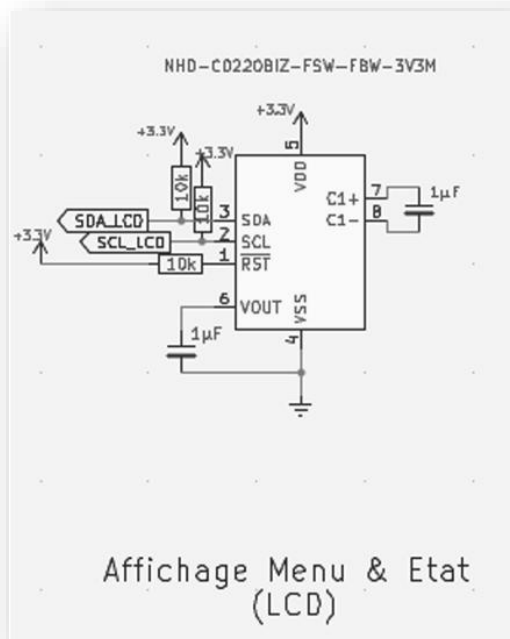
Schéma structurel

Le schéma ci-dessous présente l'unité de traitement architecturée autour du microcontrôleur PIC24FJ64GA002. Il met en évidence une distribution sectorisée et optimisée des entrées/sorties : le protocole I2C pilote l'affichage textuel, le bus SPI (remappé sur les broches inférieures) gère la matrice RGB du ruban, tandis que les boutons et le commutateur exploitent les lignes d'interruption matériel CN pour maximiser la réactivité du système.

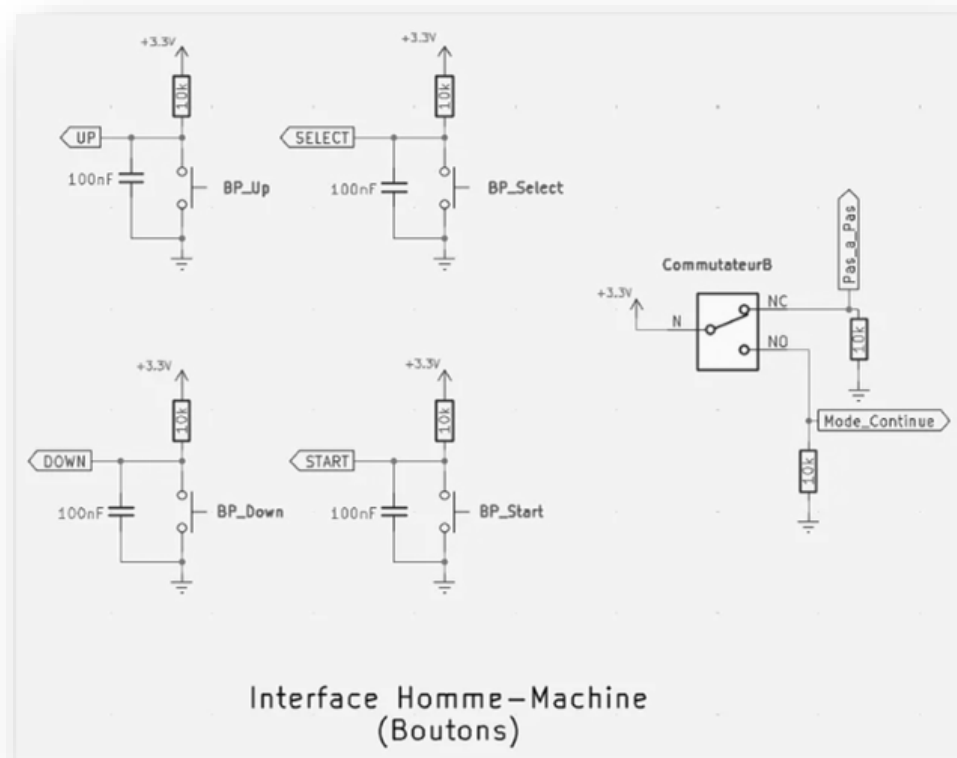


Unité de traitement
(Microcontrôleur)

Cette section du schéma ci-dessous met en évidence la sectorisation des interfaces de sortie de la machine. L'afficheur textuel LCD utilise le bus I2C (3,3V) avec des lignes de rappel à un niveau haut de 10k Ω , limitant l'utilisation des broches du microcontrôleur. En parallèle, le contrôleur de la matrice LED RGB (le ruban) est piloté via un bus SPI dédié. Ce choix du SPI synchrone garantit le débit de données nécessaire à la fluidité visuelle du ruban, tout en adaptant les niveaux de puissance puisque la matrice fonctionne sous une alimentation distincte de 5V.



Ce schéma ci-dessous détaille l'implémentation matérielle de l'interface homme-machine d'entrée. Les quatre boutons-poussoirs dédiés au contrôle de l'application intègrent des résistances de pull-up de $10k\Omega$ maintenant un niveau logique haut par défaut. À l'inverse, le commutateur à levier exploite deux résistances de pull-down de $10k\Omega$ afin de basculer de manière stable entre les lignes de commande « Pas_a_Pas » et « Mode_Continue ». Cette sectorisation électrique, directement reliée aux entrées de notification de changement (CN) du microcontrôleur, élimine les risques de rebonds logiciels critiques lors de l'exécution des tables de transition.

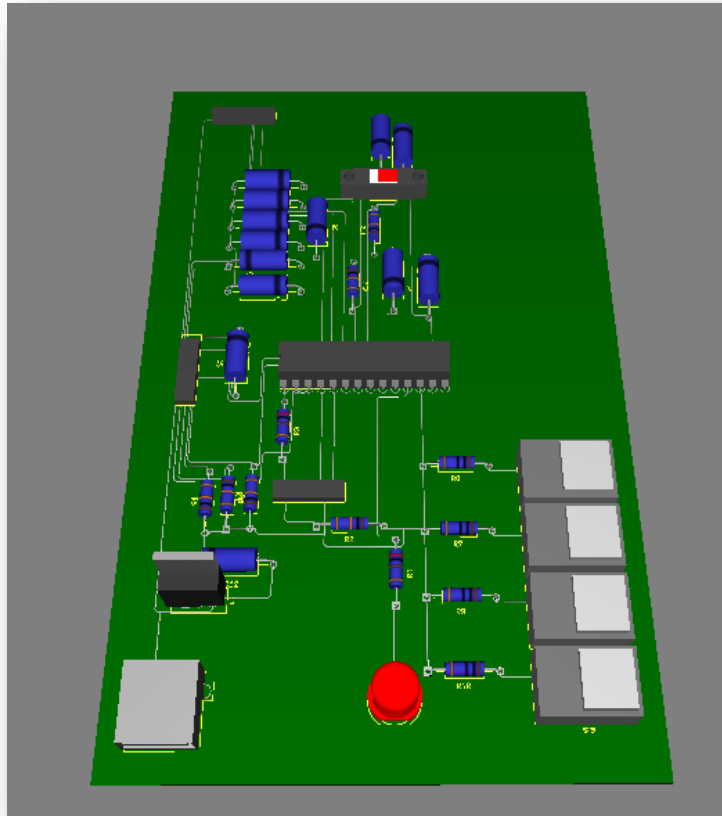


Conception du circuit imprimé (PCB)

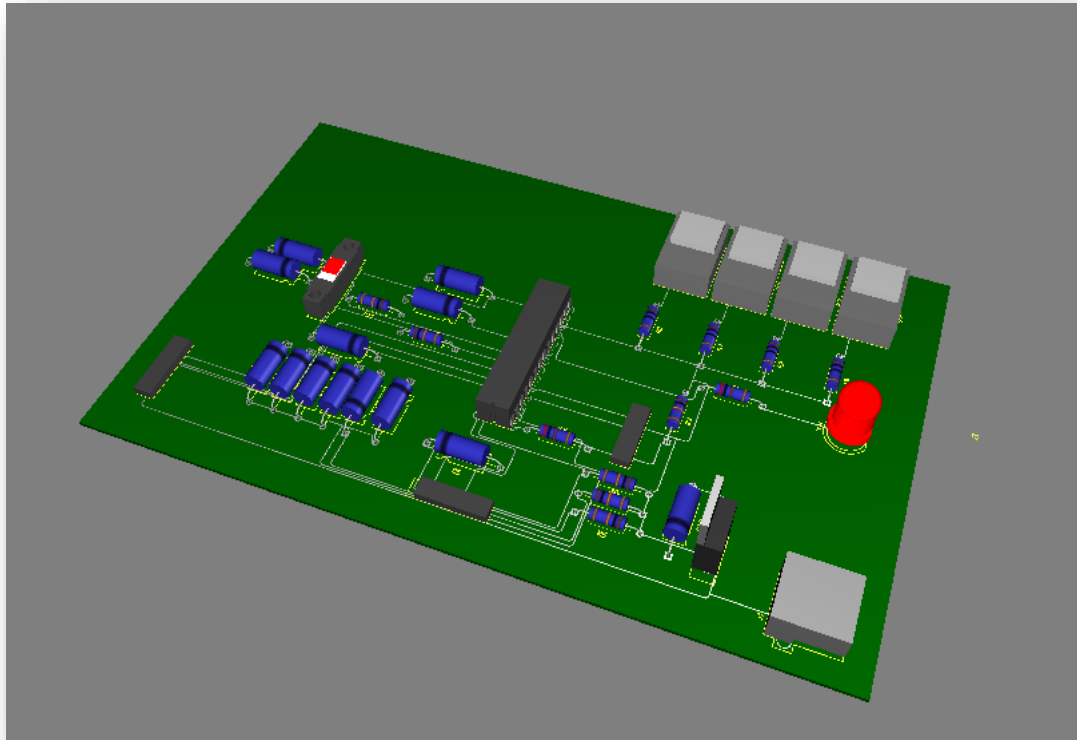
À l'issue de la phase de validation matérielle sur platine d'essai, nous avons procédé au routage d'un circuit imprimé (PCB) dédié. L'objectif principal de cette étape est de fiabiliser les connexions électriques, d'éliminer les risques de faux contacts inhérents au prototypage, et de compacter l'électronique en vue de son intégration dans le boîtier final.

Le design de cette carte a été pensé pour centraliser l'unité de traitement, la distribution d'énergie, ainsi que le routage optimisé des bus de communication (SPI et I2C) vers les périphériques externes (matrice LED et écran LCD).

Les figures ci-dessous présente le rendu de notre conception avant fabrication :



Modélisation 3D de la carte électronique – Vue de dessus



Modélisation 3D de la carte électronique – Vue isométrique

VI. Conception logicielle et algorithmique (Software)

Logique de traitement

Afin de valider le fonctionnement logique de notre Machine de Turing, nous avons implémenté un algorithme d'addition de deux nombres binaires (voir ci-dessous). Cet algorithme repose sur une machine à états finis de l'état 1 à 6 permettant de parcourir la bande, de décrémenter le second opérande et d'incrémenter le premier par propagations successives de retenues. La table des transitions complète, détaillant les états, les lectures de couleurs (Rouge pour 0, Vert pour 1, Vide pour le séparateur) et les déplacements de la tête de lecture, est disponible en Annexe de ce rapport.

```

void addition(Etat *Etats) {

    // ETAT 0 : Chercher la fin de la bande tout a droite
    Etats[0].final = 0;
    action_create(&Etats[0].transition[1], 1, droite, 0); // Si 1 -> on avance
    action_create(&Etats[0].transition[2], 2, droite, 0); // Si 0 -> on avance
    action_create(&Etats[0].transition[0], 0, gauche, 1); // Si Blanc -> on recule et on passe a
l'Etat 1

    // ETAT 1 : Soustraire 1 au nombre B (Droite)
    Etats[1].final = 0;
    action_create(&Etats[1].transition[2], 1, gauche, 1); // Lit 0 (Rouge) -> Ecrit 1 (Vert),
propage a gauche
    action_create(&Etats[1].transition[1], 2, gauche, 2); // Lit 1 (Vert) -> Ecrit 0 (Rouge), fin
de soustraction (Etat 2)
    action_create(&Etats[1].transition[0], 0, rien, 6); // Lit Blanc -> B est vide ! On a fini
l'addition (Etat 6)

    // ETAT 2 : Voyager a gauche jusqu'au deuxieme nombre (A)
    Etats[2].final = 0;
    action_create(&Etats[2].transition[1], 1, gauche, 2); // Si 1 -> on recule
    action_create(&Etats[2].transition[2], 2, gauche, 2); // Si 0 -> on recule
    action_create(&Etats[2].transition[0], 0, gauche, 3); // Si Blanc (separateur) -> on passe
le separateur (Etat 3)

    // ETAT 3 : Ajouter 1 au nombre A (Gauche)
    Etats[3].final = 0;
    action_create(&Etats[3].transition[1], 2, gauche, 3); // Lit 1 (Vert) -> Ecrit 0 (Rouge),
propage la retenue a gauche
    action_create(&Etats[3].transition[2], 1, droite, 4); // Lit 0 (Rouge) -> Ecrit 1 (Vert), fin
d'addition (Etat 2)
    action_create(&Etats[3].transition[0], 1, droite, 4); // Lit Blanc (debordement) -> Ecrit 1
(Vert), fin d'addition (Etat 4)

    // ETAT 4 : Voyager a droite jusqu'au separateur
    Etats[4].final = 0;
    action_create(&Etats[4].transition[1], 1, droite, 4); // Si 1 -> on avance
    action_create(&Etats[4].transition[2], 2, droite, 4); // Si 0 -> on avance
    action_create(&Etats[4].transition[0], 0, droite, 5); // Si Blanc (separateur) -> on passe le
separateur (Etat 5)

    // ETAT 5 : Voyager a droite jusqu'a la fin du nombre B
    Etats[5].final = 0;
    action_create(&Etats[5].transition[1], 1, droite, 5); // Si 1 -> on avance
    action_create(&Etats[5].transition[2], 2, droite, 5); // Si 0 -> on avance

```

```

    action_create(&Etats[5].transition[0], 0, gauche, 1); // Si Blanc (fin de bande) -> on
    recule et on boucle a l'Etat 1

    // ETAT 6 : Etat final
    Etats[6].final = 1; // La machine s'arrete ici !
}

```

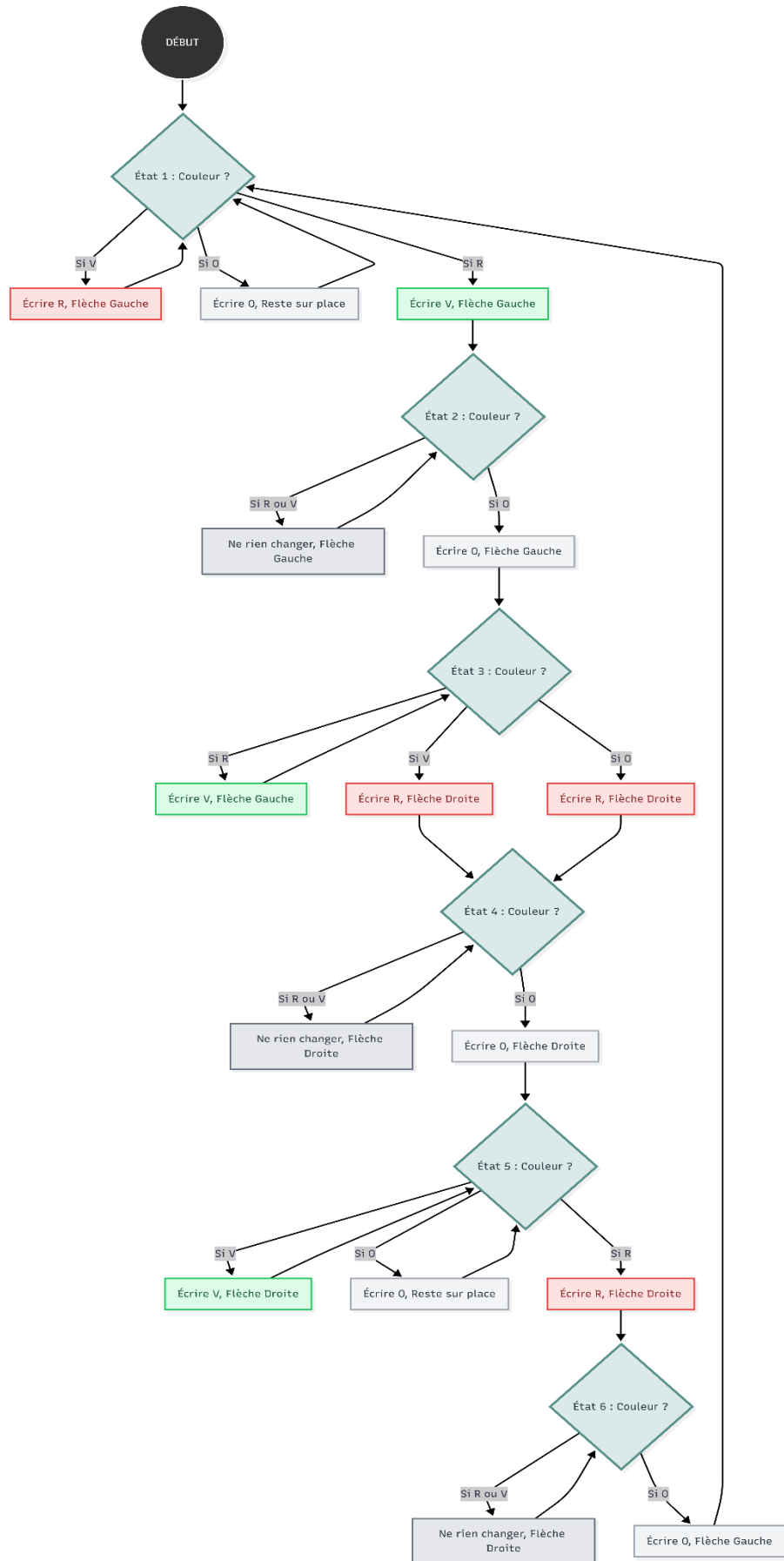
Mod lisation algorithmique

Pour illustrer le comportement de l'algorithme impl ment  dans le microcontr leur, la figure ci-dessous mod lise l'encha nement des  tats sous forme de logigramme.

Le principe fondamental de cette machine    tats repose sur l' valuation et la modification des couleurs lues sur le ruban matriciel.

L'algorithme boucle sur une s quence spatiale stricte : la machine  value la couleur actuelle ( tat 1) et l'inverse selon les r gles de calcul, puis la t te de lecture se d place vers la gauche ( tat 2) pour aller reporter la retenue ( tat 3). Une fois la couleur mise   jour, la t te repart vers la droite ( tats 4 et 5) pour traiter le symbole suivant.

Les  critures mat rielles sur la matrice LED sont instantan es et ce cycle de balayage (gauche/droite) se r p te inlassablement jusqu'  la d tection d'une case vide (symbole O) lors de l' tat 1, ce qui d clenche l'arr t d finitif de la machine ( tat 6).



VII. Intégration et Validation Expérimentale

Protocoles de Test

Afin de garantir le fonctionnement fiable et robuste de la Machine de Turing, nous avons établi une campagne d'essais méthodique. L'objectif de cette phase était de valider physiquement et fonctionnellement chaque bloc de notre architecture avant leur assemblage final.

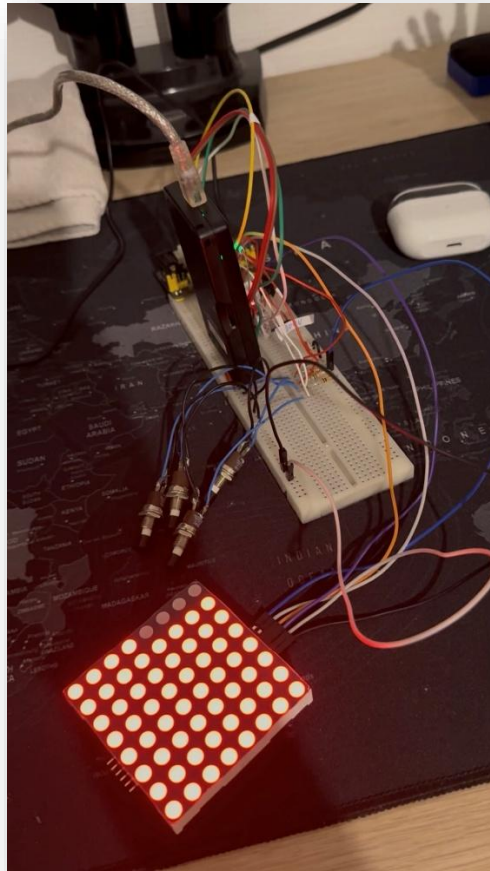
Elément Validé :

- Reconnaissance et programmation : Le microcontrôleur PIC24FJ64GA002 est correctement reconnu. La programmation est pleinement opérationnelle.
- Entrées numériques : Les entrées sont bien détectées par le programme, ce qui valide la partie matérielle pour les deux pins concernées (boutons poussoirs "UP" et "DOWN").
- Logique de calcul : Le programme lance correctement la procédure de calcul (addition) pour donner suite à l'action sur les boutons "START" et "SELECT".
- Gestion de la matrice (Ruban RGB) : La matrice est bien pilotée via le bus SPI, validant ainsi la gestion séquentielle des cases du ruban.
- Signalisation d'erreur : Le circuit de diagnostic matériel est opérationnel, la LED rouge d'erreur s'allume correctement via le code.

Elément non validé :

- Communication I2C (Écran LCD) : L'écran reste totalement inerte. On suspecte un défaut matériel.
- Bascule de mode (Commutateur) : Bien que le câblage soit validé et que le signal soit stable sur l'entrée lors du basculement, le test n'a pas été réalisé.

La fiche de test est disponible en Annexe.



Prototypage sur platine d'essai validant le pilotage de la matrice LED par le microcontrôleur

Elaboration des codes de test unitaire & firmware final

Pour mener à bien nos protocoles de test, nous avons opté pour une approche de développement modulaire. Plutôt que de programmer l'ensemble du système d'un seul bloc, nous avons mis en place des codes de test unitaires pour chaque périphérique.

Nos objectifs,   travers la mise en place de ces scripts de validation,  taient les suivants :

- Code de test des Interruptions (Entr es) : L'objectif  tait de configurer et de valider le module *Change Notification* (CN) du PIC24. Nous voulions nous assurer qu'une action physique de l'utilisateur d clenche instantan ment une routine d'interruption logicielle, confirmant ainsi que le microcontr leur n'a pas besoin de perdre du temps de calcul   scruter l' tat des boutons.
- Code de test de l'Afficheur LCD : Le but  tait d'isoler la communication I2C. Le code mis en place visait uniquement   initialiser le bus, cibler l'adresse mat rielle de l' cran, et afficher une cha ne de caract res simple (ex: "Test OK").
- Code de test de la Matrice RGB : Ce code r pondait   un double objectif. Prem rement, valider le param trage logiciel du registre PPS (Peripheral Pin Select) permettant d'assigner les broches physiques aux signaux SPI. Deuxi mement, envoyer des s quences de couleurs pures (Rouge, Vert, Bleu) pour s'assurer de la synchronisation de l'horloge SPI (SCK) et de l'int grit  de la ligne de donn es (MOSI).
- Code de test de la LED d' tat : L'objectif de ce script tr s court  tait de v rifier la bonne configuration des directions des ports et de confirmer que l'oscillateur principal du syst me fonctionnait   la fr quence attendue.

Ce n'est qu'une fois ces objectifs logiciels unitairement atteints que nous avons pu fusionner ces codes pour former le firmware complet de la machine.

L'ensemble du code est disponible en Annexe.

Conclusion générale et perspectives

L'aboutissement de ce projet marque la transition réussie d'un modèle mathématique théorique vers un prototype matériel interactif. Notre mission initiale visait à stabiliser les travaux des promotions précédentes pour livrer une Machine de Turing électronique exploitable et fiable.

Si l'on se réfère à notre cahier des charges, l'objectif critique est rempli : l'ensemble des fonctionnalités critiques ont été implémentées avec succès. L'architecture matérielle est aujourd'hui stabilisée avec la conception et le routage finalisé d'un circuit imprimé (PCB), bien que ce dernier n'ait pas encore été envoyé en fabrication. Les communications avec les périphériques fonctionnent, et l'exécution du programme d'addition binaire valide le cœur logique du système.

Face aux impératifs d'intégration matérielle et temporelle, nous avons fait le choix de privilégier la fiabilité du système au détriment du nombre de fonctionnalités. De ce fait, les interfaces d'usage secondaires, telles que la commutation des modes d'exécution (pas-à-pas et continu) ainsi que l'affichage textuel sur l'écran LCD, n'ont pas encore été totalement finalisées.

Si le montage actuel prouve la viabilité du concept, il dessine également une feuille de route claire pour les futures itérations de la machine.

Le socle logique et l'architecture électronique étant validés, les développements futurs devront se concentrer sur l'achèvement de l'interface utilisateur et la finalisation matérielle de la machine. Nous suggérons d'orienter les futures itérations autour des axes suivants :

- Programmer l'affichage textuel sur l'écran LCD pour indiquer l'état courant. Traiter les interruptions matérielles pour rendre fonctionnelle la commutation entre le mode d'exécution continu et le mode pas-à-pas.
- Créer une routine logicielle permettant d'entrer de nouvelles tables de transition directement via les boutons. Cela élargira les opérations possibles (soustraction, multiplication) sans nécessiter de reprogrammer le microcontrôleur.
- Lancer la fabrication physique du circuit imprimé (PCB). Modéliser et imprimer un boîtier en 3D pour protéger l'électronique.

Remerciement

Nous tenons à exprimer notre gratitude à l'ensemble des personnes qui ont contribué au bon déroulement de ce projet.

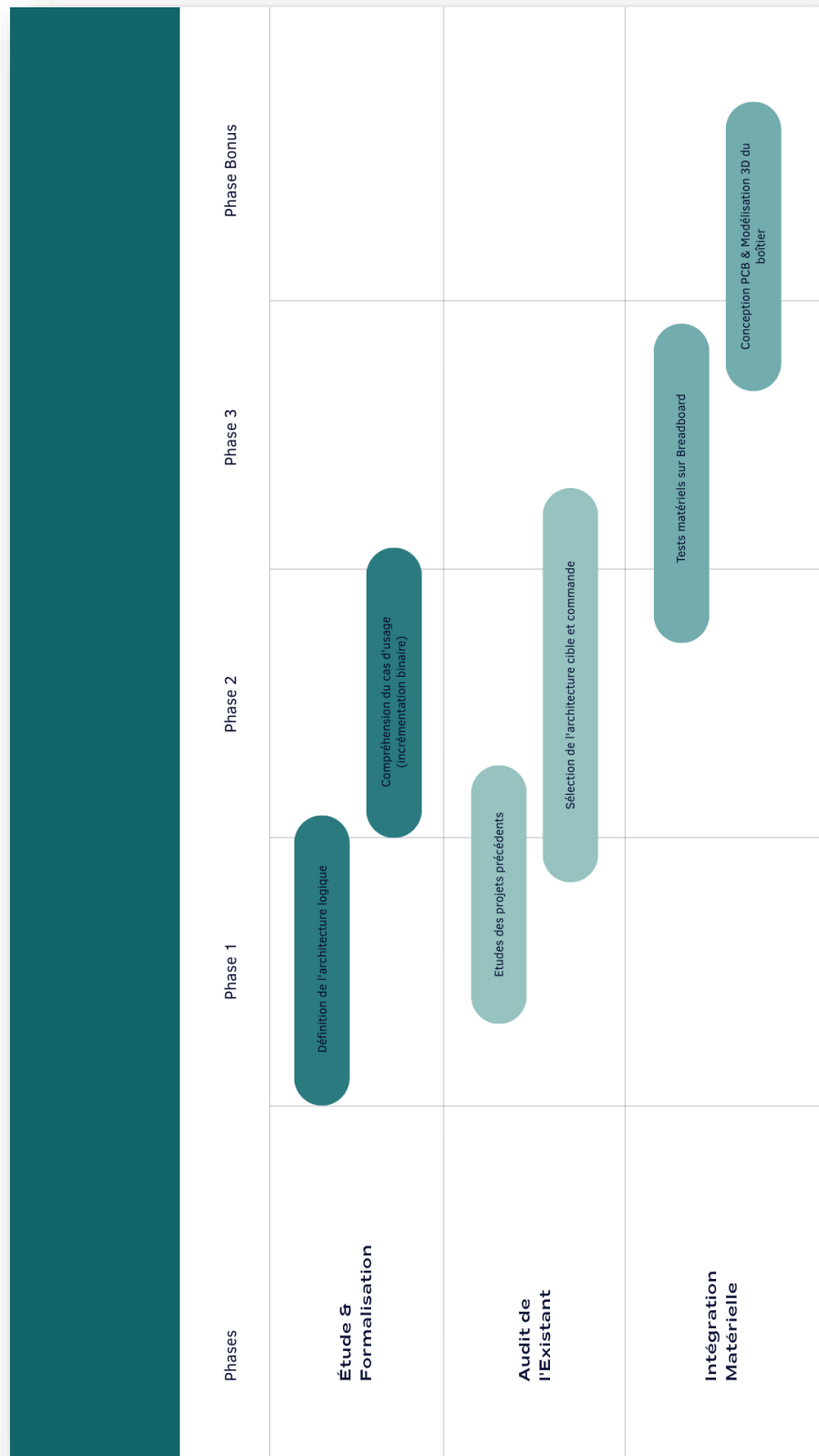
Nos remerciements s'adressent tout d'abord à Monsieur Bocquillon, notre tuteur, pour sa confiance, son suivi et ses conseils tout au long de la réalisation de ce travail. Son accompagnement direct a été un élément moteur dans l'atteinte de nos objectifs.

Nous saluons également l'implication de Monsieur Rolland pour son expertise technique, sa disponibilité et ses orientations méthodologiques indispensables lors des phases clés de conception du système.

Enfin, nous tenons à remercier tout particulièrement Monsieur Clément ALLEGRE-COMMINGES et Monsieur Romain BROUARD. Leur intervention lors de la réunion de cadrage initiale a été déterminante pour définir les contours du projet, comprendre les attentes pédagogiques du système et poser les bases de notre réflexion globale.

Annexe & bibliographie

Feuille de route du projet



Algorithme d'addition de deux nombres binaires

Etat initial de l'afficheur

			R		R		R				R	R			
				V		V			V	V					
O	O	O						O					O	O	O
○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○

Programme de l'algorithme

Etat Actuel		Etat Futur		
N° d'état	Couleur	Couleur	Flèche	N° d'état
1	R	V	←	2
	V	R	←	1
	O	O		1
2	R	R	←	2
	V	V	←	2
	O	O	←	3
3	R	V	←	3
	V	R	→	4
	O	R	→	4
4	R	R	→	4
	V	V	→	4
	O	O	→	5
5	R	R	→	6
	V	V	→	5
	O	O		5
6	R	R	→	6
	V	V	→	6
	O	O	←	1

Fiche de test

Système ou Objet Technique	Machine de Turing Électronique
Référence du Test	
Objectifs du test	Valider la communication avec le microcontrôleur, l'alimentation des périphériques et l'acquisition des commandes IHM de base

Préparation du test		
Matériel de <u>mesure</u> à mettre en œuvre	Matériel	Caractéristiques principales
	Alimentation	Module d'alimentation stabilisé (réglé sur 3,3V et 5V)
	Oscilloscope	X
	Multimètre	Vérification de la continuité et de la tension sur la pin Vcap
	GBF	X
Autres matériels à mettre en œuvre	Autre(s)	Sonde de programmation PICkit 4
		X
Outils spécifiques		X

Besoins logiciels	MPLAB X
--------------------------	----------------

Description du test		
Etat initial		
Opérations à effectuer	Résultat(s) attendus	Remarques / Précisions
Connecter le PICkit 4 et alimenter la breadboard	Détection du PIC24FJ64GA002 par MPLAB X	Vérifier le condensateur de 10µF sur Vcap
Actionner les boutons poussoirs "UP" et "DOWN"	Allumage des LEDs ou changement d'état logiciel	Validation de l'entrée numérique
Envoyer une trame I2C vers l'afficheur LCD	Affichage du texte de bienvenue (Hello World)	Test de la communication bus I2C
Actionner les boutons "START" et "SELECT"	Lancement de la procédure de calcul (Addition)	Vérification du multiplexage des pins
Piloter la matrice via le bus SPI	Allumage séquentiel des cases du ruban	Test de la gestion du ruban RGB
Basculer le commutateur entre les deux modes	Changement d'état détecté par le microcontrôleur	Vérification du signal sur la pin RP11
Forcer une condition d'erreur logicielle	Allumage de la LED rouge de diagnostic	Test du circuit de signalisation d'erreur

Déroulement du test		
Positionnement à l'état initial		
Résultat(s) obtenus	Opération Validée	Remarques / précisions
PIC24FJ64GA002 reconnu	Validé	Programmation opérationnelle
Les entrées sont bien détectées par le programme	Validé	Hardware validé pour ces deux pins
L'écran reste inerte	Non Validé	Défaut au niveau du code ou LCD HS
Le programme lance bien la procédure de calcul	Validé	
La matrice est pilotée via le bus SPI	Validé	
Câblage OK, signal stable sur l'entrée	Non Validé	Test logiciel à réaliser pour confirmer la bascule Continu/Pas-à-pas
La LED s'allume correctement via le code	Validé	Hardware opérationnel pour le retour d'erreur

Firmware de la machine de Turing

Disponible via ce lien GitHub : <https://github.com/lilDyNo94/Projet-machine-de-Turing.git>

Lien :

<https://machine-de-turing.fr/algorithmes.php>

<https://www.youtube.com/watch?v=P66h8D5Lkwk>

https://github.com/sparkfun/LED_Matrix_Serial_Interface_RGB/tree/master